

Odoo Sidekick — Technical Manual

A Claude skill by SHIFTcollective, bridging Claude to Odoo 19 over the External JSON-2 API.

Version covered: 1.9 **Last updated:** 2026-08-18

This is the in-depth technical reference: setting the skill up, understanding its safety model in full, and getting the most out of reads, writes, caching, and automation, aimed at admins, implementers, and technical/power users. **If you're a day-to-day Odoo user who just wants to get value out of this without the implementation details, read `quick-start-guide.md` instead** — it's six pages and covers everything you actually need. For the underlying agent instructions the skill runs on, see `SKILL.md` ; for narrower technical references (domain syntax, model field lists, Odoo 19 schema quirks), see the `references/` directory this manual points to throughout.

Contents

1. [What This Skill Does](#)
 2. [Getting Started](#)
 3. [First-Time Setup Walkthrough](#)
 4. [Understanding the Safety Model](#)
 5. [Reading and Analyzing Data](#)
 6. [Making Changes: Writes and Confirmation](#)
 7. [Caching for Heavy Analysis](#)
 8. [Automation and Multi-Agent Use](#)
 9. [Troubleshooting](#)
 10. [Command Reference](#)
 11. [Tips and Best Practices](#)
 12. [Licensing](#)
 13. [Getting Help](#)
-

What This Skill Does

Odoo Sidekick is a Claude skill that bridges Claude to an Odoo 19 instance over Odoo's External JSON-2 API (`POST /json/2/<model>/<method>`). It lets you ask Claude, in plain language, to pull data out of Odoo, analyze it, and, if you choose to allow it, make changes back to it. There's no separate app to run: the skill lives in your skills directory and Claude invokes its Python scripts directly.

By default every profile is read-only. You can query sales orders, invoices, inventory moves, manufacturing orders, contacts, and purchases using a whitelist of safe read methods (`search_read` , `read_group` , and similar), and optionally mirror any model into a local DuckDB or SQLite cache for faster, repeated analysis (aging reports, cohort breakdowns, joins across models that would be slow to re-fetch every time). Writes are opt-in: a profile has to be explicitly configured as `read-write` , and even then every request that implies a write (creating a contact, updating a price, posting an invoice) requires explicit confirmation, either from you in chat or from an operator-configured approval mechanism in unattended deployments. Nothing is written without an approved plan. Since v1.9 the skill can also learn

who it's talking to — role, company size, goals, decision cadence — via a few skippable questions (or a headless seed file) and tailor its default reports, depth, and phrasing accordingly; see **The user profile** in the setup chapter.

The skill runs from three different Claude surfaces, each with different runtime characteristics — Claude.ai (web/mobile, sandboxed and ephemeral, good for one-off questions or demos), Cowork (your real filesystem, persistent config, good for recurring reports and local-file workflows), Claude Code (your real filesystem, persistent config, good for iterating on the skill or scripting batch operations) — plus a declared headless mode (`OD00_SIDEKICK_HEADLESS=1`) for autonomous agents and CI where no human is present to confirm anything. Profiles and cached databases stay local to whichever machine set them up.

When to use it, in your own words to Claude:

- "Show me last quarter's revenue by product category."
- "Who are our top 10 customers by sales this year?"
- "Give me an AR aging report from Odoo."
- "What's our MRP demand for next month?"
- "Audit our SKUs for missing barcodes or costs."
- "Pull this data into a spreadsheet I can take with me."
- "Create a contact for this new vendor." (requires a read-write profile and your confirmation)
- "Update the price on this product." (same)
- "Cache our sales orders locally so I can slice them repeatedly without hitting the API every time."

If you're only exploring or reporting, Claude.ai or Cowork with a read-only profile is all you need. Writes and scheduled/unattended use require deliberate setup, covered elsewhere in this manual.

Getting Started

Pick your surface

Odoo Sidekick runs on three interactive Claude surfaces, plus a declared headless mode for unattended use, but they don't all behave the same way underneath. The skill code is identical everywhere; what changes is whether the filesystem is sandboxed and whether your profile config survives past this conversation.

Surface	Filesystem	Config persists?	Best for
Claude.ai (web/mobile)	Sandboxed container	No, ephemeral per conversation	One-off queries, exploration, demos, mobile use
Cowork	Your real filesystem	Yes	Recurring reports, local-file workflows (Excel, PDFs), scheduled work
Claude Code	Your real filesystem	Yes	Skill iteration, batch operations, automation, git workflows
Headless (<code>OD00_SIDEKICK_HEADLESS=1</code>)	Operator-managed	Yes	Scheduled heartbeats, multi-agent platforms, CI, no human present at call time

A quick rule of thumb from SKILL.md and README.md: if you just want "show me last quarter's revenue" answered once, Claude.ai is fine. If you want a report you run every morning, or you're integrating Odoo data with files already on your machine, set up on Cowork or Claude Code instead, since only those two keep your profile config (and any local cache database) around between sessions. Headless is its own category, not a fallback for the other three, and is meant for autonomous agents or schedulers where no one is present to answer a confirmation prompt in chat.

Regardless of which surface you start on, remember that **the skill travels, the profile config does not**. Installing Odoo Sidekick once makes it available everywhere, but if you set up a profile in Claude.ai and later switch to Cowork, you need to redo the profile setup there (or copy the YAML file over manually).

Install

1. Copy this folder into your skills directory (or package it into a `.skill` file).
2. Install only the Python deps you actually need. Both are optional:
 - `pip install pyyaml`, only if you want to author profiles as YAML. JSON profiles work with zero extra installs.
 - `pip install duckdb`, only if you want the DuckDB cache backend for `cache_sync.py` / `cache_status.py`. SQLite is in the Python standard library, so the sqlite backend needs nothing extra.

On Claude.ai the sandbox typically already has these or installs them transparently. On Cowork and Claude Code you own dependency installation yourself, nothing auto-installs there.

Run the self-test first

Before configuring a profile or touching a real Odoo instance, run:

```
python3 -m scripts.selftest
```

This exists specifically to catch a broken or partial install (a missing script, an optional dependency you forgot, an env var set in one shell but not another) in seconds, before it fails confusingly mid-task. It checks, offline by default: your Python version (needs 3.9+), that all required skill files and script modules are present, that `SKILL.md`'s frontmatter description is within the platform's 1024-character limit (an oversized description silently breaks skill triggering), that every script module imports cleanly, whether the optional `yaml` and `duckdb` packages are installed (their absence is reported but doesn't fail the run), whether a profiles file already exists and parses, whether any `${ENV_VAR}` references in it are actually set in your current shell, whether the state directory is writable, and — if a user profile (`user_profile.yaml`, see **The user profile** below) exists — that it parses and validates, so a corrupt tailoring file surfaces here instead of erroring at session start. The optional-and-informational checks (`user_profile` among them) never fail the run.

Useful variants:

```
python3 -m scripts.selftest --profile main    # also test connectivity for profile "main"
python3 -m scripts.selftest --json           # machine-readable output
```

`--profile <name>` adds a live connectivity check (a `search_count` on `res.partner`) using that profile, so run it plain first and only add `--profile` once you actually have one configured. Whatever goes wrong during that check — a client error or something entirely unexpected — is reported as a failed `connectivity` check with the exception named, never as a crash: the self-test must survive exactly the failures it exists to diagnose. Exit code is `0` when

every required check passes and `1` if something required is broken, which makes it easy to gate automation on it (`python3 -m scripts.selftest || exit 1`). A representative clean run looks like this:

```
[ok ] python          3.11.15
[ok ] skill_files     all present
[ok ] frontmatter_description 1019/1024 chars
[ok ] imports         all scripts import
[ok ] optional_yaml   installed
[-- ] optional_duckdb  not installed – only needed for the DuckDB cache backend (sqlite works
without it)
[ok ] core_module     odoo_client imports
[-- ] profiles_file   none at ~/.config/odoo-sidekick/profiles.yaml – first-run onboarding will
create it
[ok ] state_dir       ~/.config/odoo-sidekick is writable
[-- ] user_profile    none at ~/.config/odoo-sidekick/user_profile.yaml – tailoring not captured
(optional; seed with scripts.user_profile)
[-- ] env_overrides   none set

All required checks passed.
```

The `--` lines are informational, not failures, missing `duckdb` and a not-yet-created profiles file are both fine at this stage.

For a deploy pipeline that needs to assert one *specific* thing, the aggregate `0 / 1` exit code is too blunt — it stops discriminating the moment any unrelated check goes red. Two flags narrow it. `python3 -m scripts.selftest --list` prints every check name (`python`, `imports`, `state_dir`, `connectivity`, ...). `python3 -m scripts.selftest --only imports,state_dir` restricts the verdict and the exit code to just those checks: exit `0` only if every named check is *asserted as passing* (`ok` is true), exit `1` if a named check actively fails, and exit `2` if a named check can't be asserted — either it wasn't emitted (a typo, or a stage that bailed out earlier) or it's one of the informational `--` checks that has no pass/fail answer (`optional_yaml` when PyYAML isn't installed, `profiles_file` before onboarding, `env_overrides`). So `--only optional_yaml` is a real gate on PyYAML being present, and an informational check never silently greens a gate. `--only` accepts only names from `--list`; an unknown name is a usage error (exit `2`).

Check which environment you're in

Once the self-test is clean, run the environment detector so you know what's actually possible on this surface before you start configuring anything:

```
python3 -m scripts.detect_env          # human-readable
python3 -m scripts.detect_env --json   # machine-readable, for scripts/agents
```

It reports the detected surface (`claude.ai`, `cowork`, `claude_code`, `local`, or `headless` if `ODOO_SIDEKICK_HEADLESS` is set), whether the filesystem is sandboxed, where the profile config path resolves to, whether that config will persist, and surface-specific recommendations and "best for" guidance. `local` is a catch-all, not a fourth surface you deliberately pick: it's what `detect_env` reports when it can't find any Claude.ai, Cowork, or Claude Code markers at all, and it recommends treating that case exactly like Cowork or Claude Code (persistent config, Path A for the API key). This matters most on Claude.ai: if `detect_env` reports an ephemeral, sandboxed

filesystem and you're planning recurring work, that's your signal to stop and set up on Cowork or Claude Code instead, rather than losing a freshly-built profile the moment the conversation ends.

First-Time Setup Walkthrough

The first time you trigger Odoo Sidekick in a chat session (Claude.ai, Cowork, or Claude Code), it checks whether a profile config already exists at `~/.config/odoo-sidekick/profiles.yaml` (or wherever `ODOO_PROFILES_PATH` points). If it doesn't, you get a one-time, conversational setup walk-through. It takes a few minutes and ends with a verified connection to your Odoo instance. Here's what to expect, step by step. (A note on paths: everywhere this manual writes `~/.config/odoo-sidekick/...`, that is the skill's *state directory* — profiles file, call log, update-check cache, user profile and all — and the whole directory moves wholesale when `ODOO_SIDEKICK_STATE_DIR` is set. `~/.config/odoo-sidekick` is just its default location.)

1. **Attribution.** You'll see a short note that the skill is built by SHIFTcollective, with contact info. This only appears once, on first run.
2. **Environment check.** The skill runs its environment detector and tells you what surface you're on and what that means for persistence. This matters because **your profile config does not travel between surfaces** — a profile you set up in Claude.ai lives only in that conversation's sandbox, while Cowork and Claude Code write to your real filesystem and persist across sessions. If you're in Claude.ai, you'll be asked whether this is one-off work (fine to proceed here) or something you'll want to repeat later (in which case you should set up on Cowork or Claude Code instead, where the config sticks around).
3. **Mode, first.** Before anything else, you're asked how much access the skill should have:
 - **Read-only** (recommended default): it can query and analyze, but cannot create, update, or delete anything.
 - **Read-write:** it can also write, but every write still requires your explicit sign-off in chat before it happens, on top of the profile's own mode gate.

You can create both a read-only and a read-write profile against the same Odoo instance if you want to keep analysis and write access separate. Most people start read-only and add a read-write profile later.

4. **Odoo URL.** You provide your instance URL, e.g. `https://yourcompany.odoo.com`.
5. **API key, two paths.** This is the security-sensitive step, and you're given a real choice:
 - **Path A, download a `.env` file (recommended).** The skill generates a template `.env` file for you to download. You paste your API key into it yourself, locally, then load it into your shell as an environment variable. The key value is never typed into the chat at all.
 - **Path B, paste the key directly into chat.** Simpler, but the key becomes a permanent part of your conversation history, readable in exports, screenshots, or by anyone with future access to that conversation. If you choose this path, you're asked to explicitly reconfirm the risk, and at the end of the session you're reminded to rotate (revoke and regenerate) the key in Odoo.

Path A is the safer choice specifically because conversation history is durable even when the surface's filesystem isn't. An ephemeral Claude.ai sandbox still keeps a permanent chat log, so a key typed into chat outlives the sandbox it was meant for.

Either way, you'll first walk through generating the key in Odoo itself: log in, click your profile icon, **My Profile**, **Account Security** tab, **New API Key**, then copy the key (Odoo shows it only once).

6. **Profile name.** You pick a short label for this connection, such as `main` , `prod` , or a client name. This is how you'll refer to it later if you have more than one profile.
7. **Config generation.** The skill assembles the profile (URL, API key reference, mode, and a default context for language/timezone) and asks permission before writing it to `~/.config/odoo-sidekick/profiles.yaml` . It's shown to you in YAML for readability, but treat that as a starting point, not the permanent format: the first time you edit anything with `profiles.py` (see below), the whole file gets normalized to canonical JSON.
8. **Connection verification.** Two checks close out setup. First, `python3 -m scripts.selftest --profile <profile>` confirms the install is complete, the profile parses, every environment variable your config references is actually set in the current shell, and the live connection works. It names the specific piece that's broken if something fails (bad key, env var not loaded, wrong URL, JSON-2 API not enabled on your Odoo plan, and so on).
For any profile meant to be read-only, also run `python3 -m scripts.verify_profile <profile>` . This asks Odoo directly what the underlying credential can actually do, since the profile's `mode: read-only` setting is enforced by this skill's client, not by Odoo. If the credential itself can write, `verify_profile` reports it as misaligned and walks you through creating a properly restricted Odoo user instead. Once both checks pass, the skill proceeds to your original request.
9. **Optional: 60-second tailoring (skippable).** After verification succeeds and your original request has been handled, you get a one-line, one-time offer: the skill can learn who you are — your role, what you care about, how often you check the numbers — so its defaults get more relevant. Accepting means at most three quick questions (every one of them skippable); the answers land in a small user profile file described in **The user profile** below. Declining is a first-class outcome: the skill records the skip and **never re-asks in later sessions** — the offer only reopens if you bring it up yourself ("tailor to me", "update my profile"). Either way, setup is complete at this point.

Once a profile exists, don't hand-edit `profiles.yaml` (or the JSON it may be stored as). Use the `profiles` CLI, which validates the schema, takes a lock so concurrent edits on shared/multi-agent hosts don't corrupt the file, and writes atomically:

```
python3 -m scripts.profiles list                # see configured profiles,
never prints keys
python3 -m scripts.profiles show <name>         # show one profile, api_key
redacted
python3 -m scripts.profiles add <name> --url https://... --api-key-env OD00_X_API_KEY [--mode read-
only|read-write] [--database DB]
python3 -m scripts.profiles set <name> <key> <value>          # e.g. set <name> mode read-
write
python3 -m scripts.profiles set <name> write_policy --json-value '{"allow": [...], "deny_models":
[...]}
python3 -m scripts.profiles unset <name> <key>
python3 -m scripts.profiles remove <name>
python3 -m scripts.profiles set-default <name>
python3 -m scripts.profiles migrate              # rewrite a YAML-authored
file as canonical JSON
```

A few practical notes:

- `add` requires exactly one of `--api-key` (inline, discouraged, since it lands in shell history) or `--api-key-env` (stores `${VAR}` , resolved from your environment at run time).
- `set` handles scalar keys (`url` , `api_key` , `database` , `mode` , `confirm_cmd` , `require_auth_ref`) directly; structured keys (`write_policy` , `default_context`) need `--json-value` .

- The canonical on-disk format is JSON (which is still valid YAML and loads without PyYAML installed). A hand-authored or onboarding-generated YAML file is accepted too, but any `profiles.py write` (not just `migrate`) rewrites the whole file as canonical JSON; `migrate` exists for converting one over explicitly, ahead of time, while keeping a `.yaml-backup` copy of the original.
- `url` and `api_key` can never be `unset`, only replaced with `set`; `remove` refuses to delete your last remaining profile.

For the full shape of a profiles file, including a read-write profile (`main_rw`) and a headless-style profile (`ops_agent`) with `write_policy` and `require_auth_ref` set (and `confirm_cmd` shown as a commented-out example of the headless approval hook), see `assets/profiles.example.yaml` in the skill directory. It's the authoritative reference for every field a profile can carry.

The user profile: telling the skill who you are

Connection profiles say *where* the skill connects; the **user profile** (new in v1.9) says *who it's talking to*. "Show me sales" means something different to a CFO than to a production manager, and a €10k invoice is headline news in a 5-person shop but noise in a 500-person one. The user profile is a small, explicit file at `<state_dir>/user_profile.yaml` (default `~/.config/odoo-sidekick/user_profile.yaml`, moved by `ODOO_SIDEKICK_STATE_DIR`) that captures exactly that context, and nothing else: **it holds preferences, never credentials**. Like the profiles file, its canonical on-disk format is JSON (still valid YAML, loads without PyYAML), and it is only ever edited through its CLI — validated, under an advisory lock, written atomically.

What it stores. Seven optional keys, all settable independently:

Key	Values	What it drives
<code>role</code>	free-form string ≤120 chars (<code>CFO</code> , <code>Operations Director</code> , <code>ops-agent</code>)	Which playbook of default reports/models applies, and the default depth
<code>company_size</code>	a bucket (<code>solo</code> , <code>2-10</code> , <code>11-50</code> , <code>51-200</code> , <code>201-500</code> , <code>500+</code>) or a plain employee count, normalized to a bucket	What counts as "a notable amount" (materiality hint)
<code>goals</code>	list of strings; the known vocabulary <code>revenue-growth</code> , <code>cost-control</code> , <code>operational-efficiency</code> , <code>cash-flow</code> , <code>customer-retention</code> , <code>throughput</code> unlocks extra focus reports, free text is kept as-is	Extra reports/models appended to the defaults
<code>pain_points</code>	list of free-text strings	Surfaced back in guidance so answers can address them
<code>decision_cadence</code>	<code>daily</code> <code>weekly</code> <code>monthly</code> <code>quarterly</code> <code>ad-hoc</code>	The default comparison window ("vs yesterday" vs "vs prior quarter")
<code>detail_preference</code>	<code>executive</code> <code>balanced</code> <code>operational</code> (optional)	Overrides the depth the role would otherwise imply
<code>language</code>	optional string, e.g. <code>es</code>	Preferred output language for reports and summaries

The file also carries managed keys the tool writes itself (`schema_version` , `onboarding` , `created` , `updated`); those aren't settable directly. The schema is deliberately persona-agnostic — `role: ops-agent` is as valid as `role: CFO` , because in agent deployments the "user" is an agent with a role.

The CLI. Everything goes through `python3 -m scripts.user_profile` :

```
python3 -m scripts.user_profile show [--json]           # what's stored (never errors when absent)
python3 -m scripts.user_profile guidance [--json]      # profile -> concrete defaults
python3 -m scripts.user_profile set role "CFO"
python3 -m scripts.user_profile set company_size 40    # counts normalize to buckets (-> 11-50)
python3 -m scripts.user_profile set goals --json-value '["cash-flow","cost-control"]'
python3 -m scripts.user_profile set goals "cash-flow, cost-control" # comma-split convenience
python3 -m scripts.user_profile unset language
python3 -m scripts.user_profile seed --file seed.yaml [--force] # headless pre-seed (see Automation)
python3 -m scripts.user_profile skip                   # record "asked, declined"
python3 -m scripts.user_profile clear                  # forget everything
```

Exit codes: `0` ok, `1` error (validation failure, bad seed file), `2` bad arguments. Notably, `show` and `guidance` exit `0` even when no profile exists — the JSON carries `"exists": false` so machine callers can branch without treating absence as failure. Values are validated on the way in (`decision_cadence` must be one of its five values, list keys hold at most 12 entries, and so on), so a typo fails loudly at `set` time rather than corrupting the file.

What `guidance` returns, and how Claude uses it. `guidance` is the consumption interface: it turns the stored profile into concrete defaults instead of making every session re-interpret raw keys. The free-form `role` is classified into a `role_family` (finance, accounting, production, purchasing, operations, sales, marketing, executive, or general), which selects a playbook of `default_reports` (what to lead with when asked "how are we doing?") and `default_models` . On top of that it derives: `depth` (`executive` / `balanced` / `operational` — from the role family unless `detail_preference` overrides it) with matching `phrasing` rules; a `materiality_hint` scaled by `company_size` (a starting calibration for "a notable amount" in the instance's currency, not a business rule); a `comparison_window` matching `decision_cadence` ; recognized vs free-form goals; and `is_agent` — roles containing markers like *agent*, *bot*, *automation*, *headless* flag the consumer as a machine, switching phrasing to prefer `--json` outputs and structured results over conversational chrome. It also names `suggested_routines` — v2.0 routine templates that are, until v2.0 ships, names to pre-populate rather than runnable definitions. Claude runs `guidance --json` silently at session start and applies the result for the whole session; `guidance` sets **defaults, not restrictions** — an explicit ask always wins.

Skip and clear semantics. These are different operations, not synonyms. `skip` records that tailoring was offered and declined (`onboarding: skipped`), and the offer is then never repeated — `guidance` reports this state so future sessions stay quiet unless the user reopens the topic themselves. `clear` deletes the file entirely, including the skip marker, so a later session may make the one-time offer again. Either way there is exactly one offer outstanding at a time: the skill asks at most three quick questions, every answer is skippable, and `pain_points` is only captured when volunteered, never interrogated.

What the user profile can never do. `role` and everything else in this file shape *presentation* — default reports, depth, phrasing. They never widen permissions: a `role: CFO` gets no write access a read-only profile denies, and an agent-flavored role earns no exemption from a single confirmation gate. The safety model in the next section is entirely unaffected by anything stored here.

Like connection profiles, the user profile does not travel between surfaces — it lives in the state dir of whichever machine captured it.

Understanding the Safety Model

Odoo Sidekick can write to your Odoo database, so it is worth understanding exactly what stands between "Claude decided to call `create`" and "a record actually got created." There are layers, and only one of them is a real security boundary. Get this straight before you enable read-write profiles, and doubly so before you deploy the skill headlessly.

The API key is the real boundary, not the profile's `mode`

Every profile in `~/.config/odoo-sidekick/profiles.yaml` (or wherever `ODOO_PROFILES_PATH` points) declares `mode: read-only` (the default) or `mode: read-write`. `scripts/odoo_client.py` enforces this itself, in Python, before it ever opens an HTTP connection: if a profile is `read-only` and the method being called isn't in the client's read whitelist (`search`, `search_read`, `read`, `read_group`, `fields_get`, `has_access`, and a handful of others), the client raises `WriteNotAllowed` and nothing is sent.

That is a genuinely useful guard against Claude accidentally calling `create` or `unlink` on a profile you meant to be read-only. But it is a client-side gate, not a permissions system. It only binds code that goes through this client. The profile's `api_key` is a bearer token for whatever Odoo user it belongs to, and that Odoo user's actual group memberships are what determine what the key can do, full stop. A `curl` command, a different script, or a bug that bypasses `OdooClient` entirely is not restrained by `mode: read-only` in any way, because that string only exists in this skill's Python, not in Odoo.

So "read-only" as configured here is a statement of intent, not a proof. To turn it into a proof, run:

```
python3 -m scripts.verify_profile <profile>
```

This calls Odoo's `has_access` for `read / write / create / unlink` against a representative spread of models (`res.partner`, `sale.order`, `account.move`, `product.product`, `stock.picking`, `ir.model` by default, plus anything named in the profile's `write_policy.allow`) and never mutates anything. If a profile claims `mode: read-only` but the credential can actually write to any probed model, it exits 2 with a MISALIGNED verdict and tells you to bind the key to a genuinely restricted Odoo user instead. Exit 0 means mode and credential capability agree; exit 1 means it couldn't verify — connection failure, bad auth, or every probed model coming back inconclusive (e.g. none of them installed on that database) — and you should treat the credential as write-capable until you run it again. Run this at setup time and any time an API key changes; for a headless deployment, it belongs in your deploy pipeline (`verify_profile --json`), not something you remember to do manually after an incident.

The `--confirm` / `confirm=True` gate and dry-run previews

Even on a read-write profile, every write method (`create`, `write`, `unlink`, `copy`, or an arbitrary `execute(... , method=...)` like `action_confirm`) requires `confirm=True` to actually execute. Call it without that and the client raises `WriteNotConfirmed` instead of sending anything, carrying a `.preview` payload that describes exactly what would have gone out: model, method, an impact string ("Updates field(s) [phone] on 3 record(s)", "DELETES 2 record(s) — IRREVERSIBLE"), and the redacted arguments. At the CLI this is the default behavior — omit `--confirm` and you get that same preview printed to stdout, and the process exits 5.

There is a second, richer preview available on demand, `--dry-run`, which fetches the current values of the targeted records and shows a field-by-field before-after diff instead of just a static payload. It's covered in full, with a worked example, in **Making Changes: Writes and Confirmation** below; the short version is that it sends no write regardless of outcome, always exits 0, and is the artifact worth building before you ask anyone to approve a plan.

This layer exists to stop accidental writes from loops, retried scripts, or pasted code, not to stop Claude specifically. Claude is expected to never bypass it: if you see `WriteNotConfirmed`, that means chat-level authorization for this exact plan hasn't happened yet, and the correct response is to go get it, not to silently retry with `confirm=True` tacked on.

Chat-level batched confirmation

Above the mechanical gates is a behavioral rule for how Claude is supposed to use this skill in an interactive session: for each user request that implies writes, Claude plans every write the request needs, presents one consolidated summary (profile, model, method, an impact statement, and representative record ids/names for anything touching more than one record, with a warning marker on irreversible operations like `unlink`), and asks once with a yes/no/something-else poll. Only after you say yes does Claude execute the whole approved batch in sequence with `confirm=True` on each call.

The important detail here is the scope of that "yes": it covers only the current message. A fresh user request always gets a fresh plan and a fresh poll, Claude is not supposed to carry an earlier approval forward to justify a later, different write. If you ask for ten updates and then, in your next message, ask for one more, that one more gets its own summary and its own yes, even in the same conversation.

Optional narrower gates: choosing them for your situation

Beyond `mode` and `confirm`, a profile can opt into three additional, independent gates. None are required for a working read-write profile, they exist for cases where "any write, once a human says yes" is more permission than you actually want to grant.

`write_policy` narrows a read-write profile down to an explicit allowlist of model+method pairs, plus a hard `deny_models` list that wins even over the allow list. Reach for this when you're handing write access to something that shouldn't have the run of the database, most concretely: you're setting up an autonomous or headless agent (a scheduled job, a multi-agent platform integration) and you want it able to, say, post chatter messages and update `sale.order`, but never touch `account.move` or `res.users` no matter what it's asked to do. The client checks this before the confirm gate, so a disallowed call is refused (`WritePolicyViolation`, a subclass of `WriteNotAllowed`) before any preview is even generated:

```
write_policy:
  allow:
    - model: sale.order
      methods: [write, message_post]
    - model: res.partner
      methods: ["*"]
  deny_models: [account.move, res.users, ir.model]
```

`require_auth_ref: true` forces every write on that profile to carry an authorization reference, an approval ticket id or similar, passed as `--auth-ref <id>` at the CLI, `auth_ref=...` in code, or via the `OD00_SIDEKICK_AUTH_REF` environment variable. Without one, the client raises `AuthRefRequired` (CLI exit 6) before sending anything. The

reference gets recorded in the call log alongside the write, so after the fact you can tell which writes trace back to a real approval and which don't. Reach for this when you need an audit trail tying writes to specific approvals, particularly for headless/autonomous deployments where nobody was in the chat to click "yes": the reference has to name something real, in an interactive session Claude is expected to ask you for the ticket id, and in an autonomous deployment the operator's tooling should supply one from the triggering routine or ticket. Fabricating one to make the gate go away defeats the entire point of the audit trail.

`confirm_cmd` points at an external command (any shell command string) that the client runs before every single write on that profile, feeding it a JSON preview of the write (profile, model, method, impact statement, redacted args, and the `auth_ref` if any) on stdin and requiring exit code 0 to proceed. A non-zero exit (or a timeout, both surfaced as `ConfirmCmdRejected`, a subclass of `WriteNotConfirmed`, CLI exit 8) refuses the write. This is the headless replacement for a human's chat "yes": if there's no one watching the conversation, `confirm_cmd` is how an operator wires in their own approval system (a webhook to Slack, a policy engine, whatever). Reach for it when you're deploying the skill to run unattended and still want a veto point on every write; in an interactive session it layers on top of chat confirmation rather than replacing it, so a hook rejecting a write the user already approved in chat is final, Claude should report that the operator's hook declined it, not retry or work around it.

Putting the layers in order

When a write call is made in read-write mode, the client checks things in a fixed order: mode gate, then `write_policy`, then the confirm gate (`WriteNotConfirmed` if `confirm` wasn't passed), then `require_auth_ref`, then `confirm_cmd`, and only after all of those pass does it touch the network. Each layer's exception name tells you exactly which check failed:

Exception	What it means
<code>WriteNotAllowed</code>	Profile is <code>read-only</code> ; no write attempt reaches the network
<code>WritePolicyViolation</code>	Profile's <code>write_policy</code> doesn't allow this model/method (subclass of <code>WriteNotAllowed</code>)
<code>WriteNotConfirmed</code>	Write method called without <code>confirm=True</code> ; carries a <code>.preview</code> of what would have been sent
<code>AuthRefRequired</code>	Profile requires <code>require_auth_ref</code> and none was supplied
<code>ConfirmCmdRejected</code>	Profile's <code>confirm_cmd</code> hook exited non-zero or timed out (subclass of <code>WriteNotConfirmed</code>)

None of these gates, individually or together, substitute for restricting what the underlying Odoo user can actually do. The recommended posture, stated directly in the skill's own defense-in-depth guidance, is to keep doing both: bind every profile, read-only or read-write, to an Odoo user whose group memberships are as narrow as the work requires, and let the client-side gates and chat confirmation handle process safety on top of that. Odoo's ACLs bound the blast radius if something goes wrong; the layers described above bound how easily something goes wrong in the first place.

Reading and Analyzing Data

Reading is what you'll do most with Odoo Sidekick: pulling sales, invoices, inventory, manufacturing, and contact data to answer questions or feed a spreadsheet. Every read goes through the JSON-2 API via `scripts/odoo_client.py`, using one of a small set of methods. Getting these right, and following two hygiene rules, is the difference between a fast, cheap query and one that floods your context window or hammers the Odoo server.

The CLI shape

All reads (and writes) run through:

```
python3 -m scripts.odoo_client <profile> <model> <method> --json '{...}'
```

`<profile>` is the name from your `profiles.yaml` (or `default`). `<model>` is the Odoo technical model name (`sale.order`, `account.move`, etc.). `<method>` is one of the whitelisted read methods below, and `--json` carries the method's named arguments as a JSON object. Read methods never need `--confirm` — that flag (and the write-confirmation flow) applies to every non-read method: `create`, `write`, `unlink`, `copy`, and arbitrary `action_*` / `button_*` calls, invoked directly by their real method name. (There's no separate `execute` verb on the CLI — that's only a convenience wrapper in the Python `odooClient` library for calling an arbitrary method.)

The core read methods

search_read — the workhorse for pulling records with specific fields. Returns a list of dicts; many2one fields come back as `[id, "Display Name"]` pairs.

```
python3 -m scripts.odoo_client main sale.order search_read --json '{
  "domain": [["state", "in", ["sale", "done"]], ["date_order", ">=", "2025-01-01"]],
  "fields": ["name", "partner_id", "amount_total", "date_order", "user_id"],
  "limit": 100,
  "order": "date_order desc"
}'
```

read — like `search_read` but for a specific set of ids you already know (e.g. from a prior `search` or a user-supplied list). Pass the whole id list in one call.

```
python3 -m scripts.odoo_client main res.partner read --json '{
  "ids": [42, 43, 44],
  "fields": ["name", "email", "credit_limit"]
}'
```

read_group — the key method for analytics. It pushes aggregation to the server instead of pulling every row and summing in Python. Aggregates use `field:aggregate` syntax (`sum`, `avg`, `min`, `max`, `count`, `count_distinct`); date/datetime `groupby` fields accept `:day`, `:week`, `:month`, `:quarter`, `:year` granularity suffixes. Note `read_group` sorts with `orderby`, not `order` like `search` / `search_read`.

```
python3 -m scripts.odoo_client main sale.order read_group --json '{
  "domain": [["state", "in", ["sale", "done"]], ["date_order", ">=", "2025-01-01"]],
  "fields": ["amount_total:sum", "id:count"],
  "groupby": ["partner_id"],
  "orderby": "amount_total desc",
  "limit": 10
}'
```

This returns rows like `{"partner_id": [42, "Acme Co"], "amount_total": 128500.0, "id_count": 17, "__domain": [...]}` — top customers by revenue in one round trip, no client-side summing needed.

`references/analytics_patterns.md` has more worked `read_group` recipes for exactly this kind of question

(revenue by month, AR aging buckets, top-N breakdowns), and `references/api_reference.md` covers the raw JSON-2 protocol underneath all of this (endpoint shape, headers, auth) if you ever need to reason about it directly.

search_count — when you only need a number, don't pull records to count them in Python.

```
python3 -m scripts.odoo_client main res.partner search_count --json '{
  "domain": [["customer_rank", ">", 0], ["active", "=", true]]
}'
```

Returns a bare integer.

fields_get — introspects a model's fields directly over the API (useful when you want the live field list rather than the cached reference docs, or need attributes like `selection` values).

```
python3 -m scripts.odoo_client main crm.lead fields_get --json '{
  "allfields": ["stage_id", "probability"],
  "attributes": ["string", "type", "selection"]
}'
```

Returns `{"stage_id": {"type": "many2one", "string": "Stage", "relation": "crm.stage", ...}, "probability": {...}}`. Omit `allfields` to get every field on the model (verbose, but useful when exploring).

Finding the right model and fields

Don't guess technical model names. Two ways to find them:

1. **Read the reference docs first.** `references/models/*.md` covers the common areas with key fields, gotchas, and worked queries: `sales.md` (sale.order, sale.order.line, crm.lead), `accounting.md` (account.move, account.move.line, account.account, account.journal), `inventory.md` (product.template/product.product, stock.quant, stock.move, stock.picking, stock.location), `manufacturing.md`, `partners.md` (res.partner), and `purchase.md`. These are the fastest path for "what field holds X" questions on models you'll query repeatedly.
2. **Ask the live database when you're unsure or need something not covered.** `python3 -m scripts.introspect <profile> --model <name>` describes a known model's fields:

```
python3 -m scripts.introspect main --model sale.order
```

If you don't know the exact technical name, `--find` fuzzy-searches installed models by keyword (backed by a 24-hour local cache, refreshable with `--refresh`):

```
python3 -m scripts.introspect main --find "sale order"
```

A guessed model name that's wrong returns a 404 from the API — search first instead of retrying variations blind.

Domain syntax

Domains are Odoo's filter language: a list of `[field, operator, value]` clauses combined with prefix-notation logical operators (AND is implicit, `|` for OR, `!` for NOT, each consuming the next clause(s)). Full details, including dot-walking through relations and common pitfalls, are in `references/domain_syntax.md`. Two representative examples:

```
# Confirmed orders over $1000 for Spanish customers
[["state","in",["sale","done"]], ["amount_total", ">", 1000], ["partner_id.country_id.code", "=", "ES"]]

# Orders in either of two date windows (explicit OR)
[["state", "=", "sale"], "|", ["date_order", ">=", "2025-01-01"], ["date_order", "<=", "2024-01-01"]]
```

Watch for the recurring traps: Odoo serializes "no value" as `false`, not `null`; many2one filters take the id (`[ "partner_id", "=", 42 ]`), not the display name; selection fields need the technical value, not the label (check `selection` via `fields_get`); and archived records (`active=False`) are excluded by default unless you add `[ "active", "in", [true,false] ]` or pass `context: {"active_test": false}`.

Two hygiene rules that matter in practice

Always pass an explicit `fields` list, and batch instead of looping. Omitting `fields` fetches every field the user can read, including expensive computed columns on rich models like `sale.order`. And if you find yourself about to call `read` once per id in a loop, stop: pass the whole id list into a single `read` call, or better, skip straight to `search_read` with a domain and skip the id round-trip entirely. The same applies to aggregation: use `read_group` rather than pulling every row and summing in Python — one call replaces what could be dozens.

Never read `ir.attachment` file bodies inline. Reading `ir.attachment` without a `fields` list, or with `datas / raw / db_datas` in it, returns the entire file as base64 in the JSON response, sometimes 9 MB or more in a single reply. The client actively refuses this: `odoo_client.py` detects the risk and exits with status 7 unless you pass `--allow-inline-binary`. Instead, download it to disk:

```
python3 -m scripts.get_attachment main --id 12245 --out ./deck.pdf
```

This decodes the file straight to a local path and prints only metadata (name, mimetype, byte count, and a checksum comparison against Odoo's stored checksum) — never the file body. Useful flags: `--overwrite` to replace an existing output file, and `--json` for a machine-readable result. If you omit `--out`, it writes to the attachment's own (sanitized) name in the current directory.

Making Changes: Writes and Confirmation

Odoo Sidekick defaults to read-only. Writing anything to your Odoo requires a read-write profile, an explicit `--confirm` on the actual call, and (in an interactive session) your sign-off in chat. This section walks through that full lifecycle.

Switching to a read-write profile

Profiles are read-only unless configured otherwise. Check what you have with:

```
python3 -m scripts.profiles list
```

If the profile you want to write against is read-only, you have three options: switch that profile's mode, use a different profile that's already read-write, or drop the write from the request. The skill will not silently upgrade a profile for you, and won't assume which of the three you want.

To flip an existing profile to read-write:

```
python3 -m scripts.profiles set main mode read-write
```

Or add a dedicated write profile alongside your read-only one (a common pattern, since `assets/profiles.example.yaml` ships with both `main` and `main_rw` as an illustration):

```
python3 -m scripts.profiles add main_rw --url https://your-instance.odoo.com --api-key-env
ODOO_MAIN_RW_API_KEY
python3 -m scripts.profiles set main_rw mode read-write
```

Keep in mind that `mode` is enforced by the client, before any HTTP call is made, but it is a client-side gate only. The Odoo API key retains whatever rights its Odoo user actually has. Run `python3 -m scripts.verify_profile <profile>` after setup (and whenever the key changes) to confirm the credential itself is as restricted as the profile claims.

What counts as a write

Any method that isn't on the read whitelist is treated as a write in a read-write profile, and is refused outright in a read-only one. Concretely:

- `create` — creates one or more records.
- `write` — updates fields on existing records.
- `unlink` — deletes records. This is irreversible.
- `copy` — duplicates a record.
- Any other non-read method called through `execute` (`action_confirm`, `button_validate`, `action_post`, `action_cancel`, `message_post`, and so on) — anything not in the read whitelist (`search`, `search_read`, `search_count`, `read`, `read_group`, `web_read_group`, `formatted_read_group`, `name_search`, `name_get`, `fields_get`, `get_views`, `default_get`, `check_access_rights`, `has_access`, `web_read`, `web_search_read`) is treated as a write and gated by `confirm`.

The two dry-run modes

Neither of these ever sends a write. They differ in how much they tell you.

Implicit preview (no `--confirm`). Any write call made without `--confirm` prints what would have been sent and exits with code `5`. This is a cheap safety net, not a planning tool. Reading `dry_run()`'s counterpart in `scripts/odoo_client.py`, the plain `call()` path raises `WriteNotConfirmed` before touching the network whenever a non-read method is invoked without `confirm=True`, and the CLI prints the JSON preview and returns exit `5`.

Explicit `--dry-run`. This is the richer mode, and it's the one worth using before you ask a user to approve anything. Reading `OdooClient.dry_run()` in `scripts/odoo_client.py`, it:

- Fetches the current values of the target records (an actual `read` call, so it costs a real round-trip but never mutates anything).
- For `write`, produces a per-record, per-field before-to-after diff (`before_after`), including whether each field actually changed (many2one values are compared by id, not by the `[id, name]` tuple Odoo returns them as).

- For `unlink`, lists the exact records that would be deleted (`records_to_delete`).
- For `create`, shows the records that would be created (`records_to_create`), redacted the same way as everything else.
- Reports `would_be_allowed`: whether the profile's mode and `write_policy` would actually let this write through, plus a `refusal_reason` if not.
- Notes (but does not run) a configured `confirm_cmd` hook — the dry-run deliberately doesn't invoke it, since it might page a human approver, but it tells you one exists and will get the final say at execution time.

`--dry-run` always exits `0` — it's a report, not a gate. Use it to build the concrete diff you show the user before asking for confirmation.

How chat confirmation works

For any user request that implies one or more writes, the skill plans the entire batch before saying anything, then asks once. The rules that shape this, straight from the skill's safety model:

- **One consolidated plan.** All writes implied by the current message are batched, not confirmed one call at a time. You'll see a single numbered list or table: profile, model, method, and an impact statement per row (e.g. "updates 12 records", or "deletes 3 records — IRREVERSIBLE" with a warning marker on anything irreversible).
- **One yes/no.** You're asked once, with three options: proceed with the whole plan, cancel entirely, or "something else" to refine (skip one row, change a value, split the batch, etc.).
- **Scoped to the current message only.** Your approval covers exactly the plan just shown. A new message starts a fresh plan and a fresh poll — approval is never carried forward, even if you're clearly continuing the same task. If you want to run another batch of writes, expect to be asked again.

If your profile also has a `confirm_cmd` configured (an operator-set external approval hook), it runs on every write in addition to your chat "yes", interactive or not. It receives the write preview as JSON on stdin and must exit `0`. If it rejects a write you already approved, the write stays refused; that's exit code `8`, and it isn't something to retry or route around.

`--auth-ref` for tying a write to a ticket or approval

If a profile has `require_auth_ref: true`, every write on it must carry an authorization reference, typically a ticket or approval id. Supply it with `--auth-ref` on the CLI (or `auth_ref=...` in code, or the `ODOO_SIDEKICK_AUTH_REF` environment variable). It's recorded in the call log alongside the write, which is what makes approved writes distinguishable from everything else after the fact.

Without it, a write against such a profile fails with exit code `6` before anything is sent. When that happens, get a real reference (from the user, the ticket, or the routine that triggered the work) rather than inventing one to make the error go away; a fabricated reference defeats the entire point of the audit trail.

Secret redaction in previews

Every preview, whether it's the implicit dry-run, the rich `--dry-run` diff, or the JSON handed to a `confirm_cmd` hook, redacts any key whose name looks secret-shaped: anything containing `password`, `api_key`, `token`, `secret`, `private_key`, `smtp_pass`, or `passwd` (case-insensitively), recursively through nested dicts and lists (so a `vals_list` full of dicts is covered too). In the `write` before-to-after diff specifically, both the old and new value are

replaced with `<redacted>` for any such field, never just the new one. You will never see a real secret value surfaced for your approval, regardless of which preview path produced it.

End-to-end example: updating a contact's phone number

Say you ask "update Acme Corp's phone number to +1 555-0142" against the `main_rw` profile. Odoo Sidekick would first resolve the partner id (a normal read), then build the dry-run:

```
python3 -m scripts.odoo_client main_rw res.partner write --json '{
  "ids": [42], "vals": {"phone": "+1 555-0142"}
}' --dry-run
```

This prints something shaped like:

```
{
  "dry_run": true,
  "profile": "main_rw",
  "model": "res.partner",
  "method": "write",
  "impact": "Updates field(s) [phone] on 1 record(s)",
  "args": {"ids": [42], "vals": {"phone": "+1 555-0142"}},
  "would_be_allowed": true,
  "before_after": [
    {
      "id": 42,
      "display_name": "Acme Corp",
      "fields": {
        "phone": {"from": "+1 555-0119", "to": "+1 555-0142", "changed": true}
      }
    }
  ]
}
```

You'd then be shown a one-row confirmation table built from that diff, something like:

#	Profile	Model	Method	Impact
1	main_rw	res.partner	write	Sets <code>phone</code> on Acme Corp (id 42): <code>+1 555-0119</code> -> <code>+1 555-0142</code>

and asked once: proceed, cancel, or something else. On "yes, proceed", the actual write is executed:

```
python3 -m scripts.odoo_client main_rw res.partner write --json '{
  "ids": [42], "vals": {"phone": "+1 555-0142"}
}' --confirm
```

If `main_rw` also set `require_auth_ref: true`, that last command would need `--auth-ref` too, e.g. `--auth-ref TICKET-123`, and would exit `6` without it. If you'd skipped `--confirm` entirely, you'd get the same request back with exit code `5` and a preview instead of a completed write.

Caching for Heavy Analysis

For one-off questions ("what did we sell last month?"), querying Odoo live is simplest and it's what the skill does by default. But some workloads are a poor fit for repeated live API calls:

- **Repeated queries against the same data.** If you're going to ask five follow-up questions about the same sales data in one session, syncing it once and querying it locally is faster and avoids re-hitting Odoo for every refinement.
- **Cross-model joins.** Odoo's API doesn't do SQL joins. "Find products with high velocity AND low stock" needs `sale.order.line` and `stock.quant` correlated together — trivial in SQL, awkward over `search_read`.
- **Snapshotting.** For financial close or period-end reporting, you want a frozen point-in-time view, not a moving target. A cache sync is that snapshot.

As a rule of thumb (from `references/caching_strategy.md`): cache when you expect more than ~5,000 rows and plan to iterate on the query, or when the workload is a recurring dashboard. Stick to live queries for one-shot questions and anything needing up-to-the-second numbers. The cache is a one-way mirror — Odoo Sidekick never writes back to Odoo from cached data, so it works fine against read-only profiles.

Backends: DuckDB or SQLite

Both backends produce the same schema (one table per model, named exactly like the model including dots, e.g. `"sale.order"`). Pick based on what you're doing with it:

- **DuckDB** (the default) — columnar storage, fast vectorized aggregations, plays well with pandas/polars. Use it for dashboards, ad-hoc analysis, and anything with joins or window functions. Requires `pip install duckdb` — it is not in the Python standard library, and `cache_sync / cache_status` will exit with a clear error telling you to install it (or fall back to `--backend sqlite`) if it's missing.
- **SQLite** — no extra install (it's in the Python stdlib), good for portability or small/moderate datasets, slower on big aggregations.

Running a sync

```
python3 -m scripts.cache_sync main --models sale.order,sale.order.line,res.partner \
    --backend duckdb --db ./cache.duckdb --incremental
```

Relevant flags on `cache_sync` (`profile` is a required positional argument):

- `--models` (required) — comma-separated Odoo model names.
- `--backend` — `duckdb` or `sqlite`, defaults to `duckdb`.
- `--db` (required) — path to the local database file.
- `--incremental` — only fetch changed records (see below). Without it, every run is a full re-sync.
- `--page-size` — rows per `search_read` page, defaults to `1000`.
- `--include-binary` — include binary fields. Skipped by default because they bloat the cache; only pass this if you actually need file contents locally.

Progress prints to stderr per model as it syncs. A failed model doesn't abort the run — the sync continues with the remaining models, then the process exits `1` if any model failed (so a scheduled/headless run can tell a partial sync

from a clean one).

What `--incremental` does

On a plain run, `cache_sync` pulls every row for each model. With `--incremental`, it looks up the last-recorded `write_date` for that model in its own `_sync_state` tracking table and adds a domain filter of `write_date >= last_write_date`, so only records touched since the previous sync come across. The comparison is deliberately `>=`, not `>`: records modified later within the boundary second would otherwise be skipped forever, and since rows are upserted by `id`, re-fetching the boundary rows is idempotent. The first sync for a model always has to be a full sync (there's no prior state to diff against); `--incremental` on a never-before-synced model just does the same full pull and starts tracking from there. Every run — including an incremental one that finds nothing new — refreshes the model's `synced_at` timestamp, so `cache_status` reports when freshness was last *verified*, and a no-change run doesn't leave the cache looking staler than it is.

Two caveats worth knowing before you rely on this for anything precise. First, **deletions are never propagated** — the sync only ever upserts, so a record removed in Odoo stays in the cache, and (contrary to what earlier editions of this manual said) a full non-incremental re-run does **not** purge it either, because a full run is just a bigger upsert. For models where deleted rows would poison an analysis, drop the model's table (or delete the whole cache DB file) and re-sync from scratch — that is the only way to purge; `references/caching_strategy.md` covers this in more depth. Second, the sync only picks up what the API user can see, so archived records dropped from view will keep whatever version was last synced.

Checking freshness with `cache_status`

Before querying cached data, check how stale it is:

```
python3 -m scripts.cache_status --backend duckdb --db ./cache.duckdb
python3 -m scripts.cache_status --backend duckdb --db ./cache.duckdb --table sale.order
python3 -m scripts.cache_status --backend duckdb --db ./cache.duckdb --json
```

`cache_status` takes `--db` (required), `--backend` (`duckdb` or `sqlite`, default `duckdb`), an optional `--table` to filter to one model, and `--json` for machine-readable output. It reports, per cached model, row count, last-synced timestamp, and a human-readable age (e.g. `4h 23m`). If the file has no `_sync_state` table yet, it tells you the DB hasn't been populated by `cache_sync`.

How staleness drives the ask-or-not decision

This is where the skill decides for you, in practice, whether to bother you about freshness at all. Before running a query against cached data, it runs `cache_status` for the relevant model and applies these thresholds:

- **Under 1 hour old** — used silently, no prompt.
- **1–24 hours old** — you'll be asked, with "use cached" as the suggested default.
- **Over 24 hours old** — you'll be asked, but this time the suggested default flips to "refresh first."
- **Over 7 days old** — you get an explicit strong warning that the data is significantly stale, on top of the ask.

So in day-to-day use: if you synced `sale.order` this morning and ask a question ten minutes later, nothing gets asked, it just uses the cache. If it's been sitting for a day and a half, expect a quick poll ("cached data is 1d 6h old — use cached, refresh first, or something else?") before it proceeds. Once you answer, that choice sticks for the rest of

the conversation, so you won't be re-asked at every turn, only if the cache crosses into a meaningfully staler band (e.g., you said "use cached" at 2 hours old but it's now 12 hours old and it matters for the query). You can also short-circuit this by stating a standing preference ("always refresh if older than 6 hours") in the session.

Two exceptions to the interactive flow: in headless/autonomous deployments (`ODOO_SIDEKICK_HEADLESS=1`), there's no one to ask, so the same thresholds apply silently — cache under 24 hours old is used, older triggers a refresh, and past 7 days a staleness warning is emitted into the output instead of into chat. And these thresholds are defaults, not hard limits — if you tell the skill your own cadence ("this table only changes weekly, don't bug me under 3 days"), it's meant to adjust to that instead. A stored user profile does some of this tuning automatically: its `decision_cadence` calibrates how eagerly staleness is worth raising (a daily-cadence user cares about hours; a monthly-cadence user usually doesn't).

Automation and Multi-Agent Use

Everything documented elsewhere in this manual assumes a human is present in a chat, picking profiles, answering onboarding questions, and typing "yes, proceed" to write plans. When Odoo Sidekick runs from a cron job, a CI pipeline, or an autonomous multi-agent platform, none of that is true, and the skill switches to a different, explicit contract instead of guessing.

Declaring headless mode

Set one environment variable before invoking any script:

```
export ODOO_SIDEKICK_HEADLESS=1
```

`scripts/detect_env.py` checks this first, before any surface-sniffing, and if it is set to `1`, `true`, or `yes` (case-insensitive) it reports `"environment": "headless"` unconditionally, regardless of what filesystem or sandbox markers are present. This is a deliberate override: the same host can serve both interactive and scheduled runs, and the operator's declaration always wins over inference. Run it once at the top of any scheduled job so downstream logic (yours or Claude's) can branch on it:

```
python3 -m scripts.detect_env --json
```

What changes in headless mode

Per SKILL.md's "Headless / autonomous operation" section, a headless run drops every conversational behavior and replaces it with machine-checkable contracts:

- **No onboarding, no welcome-back, no attribution, no cache-staleness questions.** These are chat conventions with nothing to render to. Staleness thresholds are still applied, just silently and with fixed defaults: use cached data if under 24 hours old, refresh if older, and if the cache is older than 7 days also emit a staleness warning in the output itself rather than asking anyone.
- **Everything talks JSON.** `detect_env`, `check_updates`, `show_metrics`, `cache_status`, `introspect`, `selftest`, `verify_profile`, `get_attachment`, `profiles list / show`, and `user_profile show / guidance` all take `--json`. The client CLI (`scripts/odoo_client.py`) is a partial exception: its `--json` flag is the *input*

payload of method arguments, not an output toggle — for machine-parseable **error** output on failures, add `--json-errors` alongside it. Confirmed by `python3 -m scripts.odoo_client --help`, which shows both flags independently.

- **Exit codes are the contract.** A headless caller should branch on the process exit code rather than parsing prose. Verified directly against `_cli()` in `scripts/odoo_client.py`:

Code	Meaning	Raised when
0	OK	Call succeeded (or <code>--dry-run</code> completed)
1	Error	Generic <code>OdooClientError</code> (e.g. profile/config problems)
2	Bad arguments	<code>--json</code> payload isn't valid JSON, or isn't a JSON object
3	Refused — mode/policy	<code>WriteNotAllowed</code> (profile is read-only) or <code>WritePolicyViolation</code> (the write isn't in <code>write_policy</code> 's allowlist, or is in <code>deny_models</code>)
4	API error	<code>OdooAPIError</code> — the server rejected the call (4xx/5xx from Odoo itself)
5	Unconfirmed write preview	<code>WriteNotConfirmed</code> — a write was attempted without <code>--confirm</code> ; the would-be payload is printed to stdout and nothing was sent
6	Auth ref required	<code>AuthRefRequired</code> — the profile has <code>require_auth_ref: true</code> and no <code>--auth-ref / auth_ref / OD00_SIDEKICK_AUTH_REF</code> was supplied
7	Inline binary refused	An <code>ir.attachment.datas / raw / db_datas</code> field would return a base64 blob inline; use <code>scripts/get_attachment.py</code> instead, or pass <code>--allow-inline-binary</code> if you deliberately want it
8	<code>confirm_cmd</code> rejected	The operator's approval hook (<code>ConfirmCmdRejected</code>) exited non-zero on this write's preview, or timed out (the hook is given 120s)

Treat 3, 6, 7, and 8 as refusals, not errors to route around: each has exactly one correct response (switch approach or supply the missing input), never a retry with the same call plus a bypass flag.

Write authorization without a chat

In an interactive session, write authorization comes from a human typing "yes" to a consolidated chat-level plan.

Headless runs have no chat, so that step is replaced entirely by whatever the profile is configured with — see

Understanding the Safety Model for the full picture; the headless-relevant pieces are:

- `write_policy` bounds which model+method pairs are attemptable at all before anything else is checked.
- `require_auth_ref: true` forces every write to carry `--auth-ref <ticket-id>` (or `OD00_SIDEKICK_AUTH_REF`), which is recorded in the call log so approved writes are provably distinguishable from anything else later.
- `confirm_cmd` is an operator-configured external command that receives the write's JSON preview on stdin and must exit `0` for the write to proceed. It runs on every write, interactive or not, and in headless mode it is effectively the whole approval layer.

An example of a hardened headless profile (from `assets/profiles.example.yaml`):

```
ops_agent:
  url: https://your-instance.odoo.com
  api_key: ${ODOO_OPS_API_KEY}
  mode: read-write
  write_policy:
    allow:
      - model: sale.order
        methods: [write, message_post]
      - model: res.partner
        methods: ["*"]
    deny_models: [account.move, res.users, ir.model]
  require_auth_ref: true
  # confirm_cmd: "/opt/approvals/check-odoo-write"
```

Pre-seeding user context

The interactive tailoring questions (see **The user profile** in the setup chapter) never run headless — like all conversational chrome, there's no one to answer them. If you want a headless agent to benefit from role-aware defaults anyway, pre-seed the user profile from a file at deploy time:

```
python3 -m scripts.user_profile seed --file ops_agent_profile.yaml
```

`assets/user_profile.example.yaml` in the skill directory is the template. A minimal seed for an operations agent looks like:

```
role: ops-agent
company_size: 11-50
goals:
  - throughput
  - operational-efficiency
pain_points:
  - late component deliveries stall MOs
decision_cadence: daily
```

Every key is optional — set only what you know. `seed` validates the file (unknown keys are rejected, values are normalized exactly as with `set`), marks the profile `onboarding: seeded`, and refuses to overwrite an existing profile unless you pass `--force`, so a redeploy can't silently clobber context an agent has accumulated. The seed file may be YAML (needs PyYAML) or JSON (needs nothing — JSON is valid YAML); if it isn't valid JSON and PyYAML isn't installed, the error says exactly that and offers both ways out. `detect_env --json` in headless mode includes a recommendation to pre-seed, plus `user_profile_path` / `user_profile_exists` fields so your deploy tooling can check whether it already happened.

Consumption is the same as everywhere else — `python3 -m scripts.user_profile guidance --json` — and an agent-flavored `role` sets `"is_agent": true` in the result, which flips the phrasing guidance toward structured, `--json`-first output. No seed means generic defaults, applied silently; a headless run never prompts about tailoring, ever.

One thing you get for free: because `user_profile.yaml` lives in the state dir, the per-agent `ODOO_SIDEKICK_STATE_DIR` isolation described below isolates user profiles too — each agent in a fleet can carry its own role and goals without any extra wiring.

Running multiple agents

Two environment variables let you fan a fleet of agents out against Odoo without their audit trails or state colliding:

- `ODOO_SIDEKICK_CALLER` — an identity string (e.g. `ops-agent-3`) stamped on every call-log record for that process. On a shared host running several agents, set a distinct caller per agent so `python3 -m scripts.show_metrics --by-caller` gives you a real per-agent breakdown instead of one undifferentiated stream.
- `ODOO_SIDEKICK_STATE_DIR` — moves the default location of the call log, `introspect`'s local model-name cache, `check_updates`' cached result (`update_check.json`), the user profile (`user_profile.yaml`), and the profiles file itself (as `<state_dir>/profiles.yaml`) for that process. (This does not affect `cache_sync` / `cache_status`'s DuckDB/SQLite data cache — that always lives wherever `--db` points, independent of the state dir.) Give each agent its own state dir for full isolation, or point several agents at the same one deliberately if you want a single shared audit log (in which case, set `ODOO_SIDEKICK_CALLER` on each so entries stay attributable).
`ODOO_PROFILES_PATH` always takes precedence over the state-dir-derived path when set, so pinning it explicitly per deployment ensures scheduled runs resolve exactly the same profile config as interactive ones do.

The one rule that matters most

An agent must never fabricate an `auth_ref` or add `--confirm` just to make an error disappear. Exit 6 and exit 5 are not bugs to route around: exit 6 means go get the real ticket or approval id (from the routine's definition, the triggering event, or by asking a human if one is reachable) and pass it; exit 5 means the write hasn't been authorized yet, full stop. The same discipline applies to a `confirm_cmd` rejection (exit 8): that is the operator's hook saying no (or failing to answer in time), not a transient failure, so report it and stop rather than retrying. Treating any of these gates as an obstacle to code around rather than an answer converts the whole audit and approval layer into decoration.

What's here today vs. what's coming

There is no built-in scheduler or routine registry yet. What ships in v1.9 is the set of CLI scripts described in this manual (`odoo_client`, `cache_sync`, `show_metrics`, `cache_status`, `verify_profile`, `selftest`, `user_profile`, and friends) plus whatever external scheduler you bring — cron, a CI pipeline, launchd, Task Scheduler, or an orchestration platform's own scheduler. A registry of named, YAML-defined routines (schedule, queries, output template, alert thresholds) executed by a dedicated `routine_runner.py` is on the roadmap for v2.0 — and it will build on the user profile that v1.9 already stores: `guidance` already emits `suggested_routines` (e.g. `morning-checkin-cfo`, `operations-standup`) matched to the stored role, which today are names to pre-populate a future routine registry with, not runnable definitions. Capturing or seeding a user profile now means the v2.0 routines will have something to tailor themselves to on day one.

Troubleshooting

How the client fails (by design)

Odoo Sidekick's client never fails silently and never papers over an error. On every failed call it surfaces the server's real exception name, the real message, and — when the failure pattern is recognized — a one-line remediation hint telling you exactly what to check or which script to run next (`scripts/odoo_client.py`'s `_hint_for` generates this). Client-side refusals (a write blocked before any HTTP call was made) work the same way: you get a named reason,

not a generic failure. The rule that follows from this is simple and non-negotiable: **read the message and hint, then fix the payload or the profile — never re-issue a failed call unchanged**. A 4xx (other than 429) or a deterministic 500 (a `builtins.*`, `odoo.exceptions.*`, or `werkzeug.*` error) will fail identically every time you resend it as-is; only transient failures (429, 502/503/504, network errors) are worth a bare retry, and for reads the client already does that for you automatically.

Common problems and fixes

Problem	What you'll see	Fix
Bad or missing API key	<code>401</code> on any call	Check the profile's <code>api_key</code> . If it uses <code>\${ENV_VAR}</code> substitution, confirm the variable is actually set in this shell session: <code>echo \$ODOO_MAIN_API_KEY</code> (or run <code>python3 -m scripts.selftest --profile <profile></code> , which checks this for you).
Wrong model name	<code>404</code> , server message contains "does not exist" and "model"	Never guess a second time. Run <code>python3 -m scripts.introspect <profile> --find <word></code> to fuzzy-search the installed models, then use the exact name it returns.
Wrong field name	<code>422 / 500</code> with <code>Invalid field '<name>'</code>	Run <code>python3 -m scripts.introspect <profile> --model <model></code> to list the real fields. Odoo 19 renamed several common ones (<code>groups_id</code> → <code>group_ids</code> , <code>kanban_state</code> removed, <code>is_close</code> → <code>fold</code>) — check references/odoo19_field_changes.md .
<code>vals</code> vs <code>vals_list</code> on <code>create</code> , or <code>orderby</code> vs <code>order</code> on <code>search / search_read</code>	Used to be a 422 ("missing a required argument: 'vals_list'") or an "unexpected keyword argument" error	The client handles this for you now: <code>create</code> accepts <code>vals</code> and normalizes it to <code>vals_list</code> (wrapping a bare dict into a one-item list), and <code>search / search_read</code> accept <code>orderby</code> and rename it to <code>order</code> . You shouldn't need to work around this anymore, but if you're calling the Odoo API directly (bypassing this client), the traps still apply.
Validation error on write	<code>422</code> , or any status with <code>error_name</code> ending <code>ValidationError</code>	Odoo rejected the values against a business rule (required field missing, broken constraint, etc.). The server message names the field or constraint — fix the payload. Don't retry unchanged.
Transient server error	<code>429</code> , <code>502</code> , <code>503</code> , <code>504</code> , or a non-deterministic <code>500</code>	For reads , the client already retried automatically (exponential backoff + jitter, default up to 2 retries — tune with <code>ODOO_SIDEKICK_MAX_RETRIES</code> or the CLI's <code>--retries</code>). If you're still seeing the error, retries were exhausted; wait and try again, or check the Odoo host's status. Writes are never auto-retried (a 502/504 can mask a transaction that actually committed), so a write that errors this way needs you to check server-side whether it went through before resending.
<code>403</code> / <code>AccessError</code>	Read or write refused by Odoo ACLs	If the message names a specific field, the API user has row access but lacks field-level access to that field — drop it from <code>fields</code> and retry. Otherwise the API user's Odoo group membership needs adjusting (Settings → Users).

Problem	What you'll see	Fix
YAML-authored profiles file, PyYAML not installed	<code>ProfileError:</code> <code>Profile file at ...</code> <code>is not valid JSON,</code> <code>and PyYAML is not</code> <code>installed</code>	Either <code>pip install pyyaml</code> (after which <code>python3 -m scripts.profiles migrate</code> can convert the file to canonical JSON so PyYAML is no longer needed at all), or re-create the config with <code>python3 -m scripts.profiles add</code> , which writes JSON directly. The same applies to a YAML <code>user_profile</code> seed file: install PyYAML or write the seed as JSON (JSON is valid YAML).

Client-side refusals: exit codes

These come from `scripts/odoo_client.py`'s CLI (`python3 -m scripts.odoo_client ...`) and mean nothing was sent to Odoo — the client refused before making the HTTP call. Each has exactly one correct response:

Exit code	Meaning	What to do
3	Refused: <code>mode</code> (read-only profile) or <code>write_policy</code> forbids this model/method	The profile is doing its job. Tell the user instead of silently switching profiles. If writes are genuinely needed, either switch to a read-write profile (with the user's knowledge) or use a different profile — don't work around it.
6	Refused: profile has <code>require_auth_ref: true</code> and no auth reference was supplied	Supply a real ticket/approval id with <code>--auth-ref <id></code> (or <code>ODOO_SIDEKICK_AUTH_REF</code>), or ask the user for one if you don't have it. Never fabricate one — a made-up reference defeats the point of the audit trail.
7	Refused: inline binary read on <code>ir.attachment</code> (<code>datas / raw / db_datas</code> , or no <code>fields</code> filter at all)	Use <code>python3 -m scripts.get_attachment <profile> --id <id> -out <file></code> to decode to disk instead. (<code>--allow-inline-binary</code> exists to override this, but avoid it — the whole point is keeping multi-MB base64 blobs out of the conversation.)
8	Refused: the profile's <code>confirm_cmd</code> hook rejected the write (nonzero exit, or it timed out)	This is an answer, not a bug. Report to the user that the operator's approval hook declined the write, and stop — do not retry, and do not try to work around the hook.

For completeness, the other exit codes you'll see from the same CLI: 0 success, 1 generic client error (e.g. profile/config problems), 2 bad CLI arguments, 4 an Odoo API error (4xx/5xx passed through from the server), 5 an unconfirmed write preview (you called a write method without `--confirm` — this is the built-in dry-run, not a failure).

Odoo 19 schema drift

A meaningful share of "wrong field" and "wrong argument" errors are specific to Odoo 19's JSON-2 API and the version's own renames, not bugs in the skill. Before assuming something is broken, check `references/odoo19_field_changes.md` — it covers field renames (`groups_id` → `group_ids` on `res.users`, `kanban_state` removed from `project.task` in favor of `state`, `is_close` → `fold` on `helpdesk.stage`), the argument-name traps above, methods you can't call over JSON-2 (private `_`-prefixed methods 403), and SaaS-specific quirks (staging DB name suffixes, `common.authenticate()` returning `uid=False`).

One entry worth calling out explicitly because it's easy to miss: `message_post` **double-escapes HTML unless you pass `body_is_html: true`**. Without it, markup in `body` gets escaped into literal `<p>` text that shows up wrong in the chatter and has to be fixed by hand. Always post HTML-bearing chatter messages like this:

```
python3 -m scripts.odoo_client <profile> res.partner message_post --json '{
  "ids": [42], "body": "<p>Approved per KIN-7902</p>", "body_is_html": true
}' --confirm
```

When nothing else works

1. Re-run `python3 -m scripts.selftest --profile <profile>` (add `--json` for a machine-readable result). It checks the install is complete, the profile file parses, every `${ENV_VAR}` the profile references is actually set in the current shell, and connectivity works — and names the specific piece that's failing rather than giving a generic error.
2. Double-check environment variables are loaded in the **current** shell. A common failure mode is setting the API key's env var in one terminal session (or sourcing a `.env` file) and then running the skill from a different shell or subprocess where it was never exported — `echo $ODOO_MAIN_API_KEY` (or the equivalent for your profile's variable name) should show the real value, not blank.
3. If you're still stuck after that, reach out to SHIFTcollective at info@shiftcollective.co — they built and maintain the skill.

Command Reference

Every script is invoked as a Python module from the skill's root directory (`python3 -m scripts.<name> ...`), never as a standalone file. Run any of them with `-h / --help` to see the live signature; this reference mirrors that output.

odoo_client — the read/write API client

One-line: send a single JSON-2 API call (read or write) to an Odoo profile, with write-safety gates built in.

```
python3 -m scripts.odoo_client [--json JSON_ARGS] [--confirm] [--dry-run]
  [--auth-ref AUTH_REF] [--retries RETRIES] [--timeout TIMEOUT]
  [--json-errors] [--allow-inline-binary] [--pretty]
profile model method
```

- `profile`, `model`, `method` are positional and required (e.g. `main sale.order search_read`).
- `--json JSON_ARGS` — JSON object of method arguments; defaults to `'{}'`.
- `--confirm` — required for any non-read method; without it you get a dry-run-style preview instead of an actual write.
- `--dry-run` — for writes, fetches current values and prints a before/after diff without sending anything. It bypasses the write-refusal exit codes (3, 5, 6, 8) — if the write would actually be refused (read-only mode, `write_policy`, missing `auth-ref`), that shows up as a field in the preview, not as a nonzero exit. A bad `--json` (2) or an unresolvable profile / network failure (1) still exits non-zero as usual.
- `--auth-ref AUTH_REF` — authorization reference recorded in the call log with the write (needed when the profile sets `require_auth_ref: true`).

- `--retries RETRIES` — max retries for transient read failures (default: `ODOO_SIDEKICK_MAX_RETRIES` env var, or 2). Writes never auto-retry.
- `--timeout TIMEOUT` — HTTP timeout in seconds (default 60).
- `--json-errors` — print a machine-readable JSON error object to stdout on failures: API errors (exit 4), generic client errors (exit 1), and client-side refusals — mode/ `write_policy` refusals (exit 3) and a missing auth-ref (exit 6) — so headless callers get structured output on every refusal path, not just server errors. (Exits 5 and 8 already print a JSON preview of the refused write to stdout regardless of this flag.)
- `--allow-inline-binary` — allow inlining base64 attachment bodies (`ir.attachment` `datas / raw`); otherwise these are refused in favor of `get_attachment.py` .
- `--pretty` — pretty-print the JSON result.

Notable exit codes: 0 success, 1 generic client error, 2 bad `--json` , 3 write refused (read-only profile or `write_policy`), 4 Odoo API error, 5 write not confirmed (dry-run preview printed), 6 `--auth-ref` required but missing, 7 inline-binary read refused, 8 a profile's `confirm_cmd` hook rejected the write.

```
python3 -m scripts.odoo_client main sale.order read_group --json '{
  "domain": [["state","in"],["sale","done"]]],
  "fields": ["amount_total:sum"],
  "groupby": ["partner_id"]
}' --pretty

python3 -m scripts.odoo_client main_rw res.partner write \
  --json '{"ids": [7, 8], "vals": {"credit_limit": 5000}}' \
  --confirm --auth-ref TICKET-123
```

introspect — discover models and fields

One-line: list installed models, fuzzy-search for one, or describe a model's fields, against a live connection.

```
python3 -m scripts.introspect (--list-models | --find QUERY | --model MODEL)
  [--pattern PATTERN] [--attrs ATTRS] [--refresh] [--json]
  profile
```

`--list-models` , `--find` , and `--model` are mutually exclusive (exactly one required). `--pattern` filters `--list-models` output by an `ilike` name pattern. `--attrs` (only with `--model`) is a comma-separated list of field attributes to show, defaulting to `string,type,required,readonly,store,help,relation,selection` . `--find` uses a local 24-hour model-list cache; `--refresh` bypasses it.

```
python3 -m scripts.introspect main --find invoice
python3 -m scripts.introspect main --model res.partner --attrs type,required,relation --json
```

profiles — manage the profiles config file

One-line: the safe, validated, atomic way to read and edit `profiles.yaml` / `profiles.json` (never hand-edit the file).

```
python3 -m scripts.profiles [--file FILE] {list,show,add,set,unset,remove,set-default,migrate} ...
```

`--file FILE` overrides the profiles path (default: `OD00_PROFILES_PATH` env var, or `<state_dir>/profiles.yaml` — `~/.config/odoo-sidekick/` unless `OD00_SIDEKICK_STATE_DIR` moves it). Subcommands:

Subcommand	Signature	Notes
<code>list</code>	<code>[--json]</code>	Lists profiles (name, mode, url, flags); never prints keys.
<code>show</code>	<code>name [--json]</code>	Shows one profile with <code>api_key</code> redacted; human-readable by default, <code>--json</code> for a machine-readable object.
<code>add</code>	<code>name --url URL [--api-key API_KEY] [--api-key-env API_KEY_ENV] [--mode {read-only,read-write}] [--database DATABASE]</code>	Creates a new profile.
<code>set</code>	<code>name key [value] [--json-value JSON_VALUE]</code>	Sets one key; use <code>--json-value</code> for structured keys like <code>write_policy</code> or <code>default_context</code> .
<code>unset</code>	<code>name key</code>	Removes an optional key.
<code>remove</code>	<code>name</code>	Deletes a profile; refuses (before touching the file) to remove the last remaining one.
<code>set-default</code>	<code>name</code>	Sets the default profile.
<code>migrate</code>	(none)	Rewrites the file as canonical JSON.

```
python3 -m scripts.profiles list --json
python3 -m scripts.profiles add ops_agent --url https://acme.odoo.com --api-key-env OD00_OPS_API_KEY --mode read-write
python3 -m scripts.profiles set ops_agent require_auth_ref --json-value true
```

user_profile — capture and serve user context

One-line: stores who is using the skill (role, company size, goals, cadence) at `<state_dir>/user_profile.yaml` and derives concrete session defaults from it; preferences only, never credentials, and never a permissions widener.

```
python3 -m scripts.user_profile [--file FILE]
    {show, guidance, set, unset, seed, skip, clear} ...
```

`--file FILE` overrides the profile path (default: `<state_dir>/user_profile.yaml`, following `OD00_SIDEKICK_STATE_DIR`). Subcommands:

Subcommand	Signature	Notes
<code>show</code>	<code>[--json]</code>	Prints the stored profile. Exits <code>0</code> even when none exists (<code>"exists": false</code> in JSON).
<code>guidance</code>	<code>[--json]</code>	Derives concrete defaults: <code>role_family</code> , <code>depth</code> , <code>materiality_hint</code> , <code>comparison_window</code> , <code>default_reports</code> / <code>default_models</code> , <code>phrasing</code> , <code>is_agent</code> , <code>suggested_routines</code> . Exits <code>0</code> with <code>"exists": false</code> plus a hint when no profile is stored.

Subcommand	Signature	Notes
set	key [value] [--json-value JSON_VALUE]	Sets one key, validated and normalized (counts → size buckets, cadence enums checked); use --json-value for the list keys goals / pain_points (a plain comma-separated string also works).
unset	key	Removes one key; a no-op (exit 0) if it wasn't set.
seed	--file FILE [--force]	Pre-seeds the whole profile from a YAML/JSON file (template: assets/user_profile.example.yaml); refuses to overwrite an existing profile without --force; marks onboarding: seeded.
skip	(none)	Records that tailoring was offered and declined, so it is never offered again.
clear	(none)	Deletes the file entirely, skip marker included.

Settable keys: role, company_size, goals, pain_points, decision_cadence, detail_preference, language — all optional; see **The user profile** in the setup chapter for values and what each drives. Writes take the same advisory lock and atomic-write path as profiles.py, and the canonical on-disk format is likewise JSON (valid YAML, loads without PyYAML).

Exit codes: 0 ok (including show / guidance with no profile present), 1 error (validation failure, bad or conflicting seed file), 2 bad arguments.

```
python3 -m scripts.user_profile set role "Operations Director"
python3 -m scripts.user_profile set goals --json-value '["throughput","cost-control"]'
python3 -m scripts.user_profile guidance --json
python3 -m scripts.user_profile seed --file ops_agent_profile.yaml --force
```

verify_profile — prove what a credential can actually do

One-line: probes the live server (read-only has_access checks, never mutates) to confirm whether a profile's key is actually as restricted as its declared mode.

```
python3 -m scripts.verify_profile [--models MODELS] [--json] profile
```

--models MODELS is a comma-separated list of models to probe (default: a representative spread across common Odoo models). Exits non-zero (2) with guidance when a profile declared mode: read-only is found to hold a write-capable key ("MISALIGNED").

```
python3 -m scripts.verify_profile main
python3 -m scripts.verify_profile main --models sale.order,res.partner --json
```

get_attachment — download attachment bodies to disk

One-line: fetches an ir.attachment and writes its decoded content to a local file, so binary blobs never land in chat context.

```
python3 -m scripts.get_attachment --id ID [--out OUT] [--overwrite] [--json] profile
```

`--id ID` (required) is the `ir.attachment` id. `--out OUT` defaults to the attachment's stored filename in the current directory. `--overwrite` allows replacing an existing output file.

```
python3 -m scripts.get_attachment main --id 4821 --out ./invoice.pdf
```

cache_sync — mirror Odoo models into a local DB

One-line: one-way sync of chosen Odoo models into a local DuckDB or SQLite file for offline/cross-model analysis; it only ever calls read methods.

```
python3 -m scripts.cache_sync --models MODELS [--backend {duckdb,sqlite}] --db DB
  [--incremental] [--page-size PAGE_SIZE] [--include-binary] profile
```

`--models` (required) is a comma-separated list of Odoo model names. `--db` (required) is the path to the local DB file. `--backend` defaults to `duckdb`. `--incremental` only fetches records with `write_date >=` the last-synced boundary (inclusive so same-second edits are never lost; the by-`id` upsert makes boundary re-fetches idempotent), and every run — even one that finds no changes — refreshes the model's `synced_at`. Deletions are never propagated (upsert-only; drop the table or DB file to purge). `--page-size` defaults to 1000. `--include-binary` opts into syncing binary fields, which are skipped by default because they're huge.

```
python3 -m scripts.cache_sync main --models sale.order,sale.order.line \
  --backend duckdb --db ./cache.duckdb --incremental
```

cache_status — check cache freshness

One-line: reports which models are cached in a given DB file, when each was last synced, how stale it is, and row counts.

```
python3 -m scripts.cache_status --db DB [--backend {duckdb,sqlite}] [--table TABLE] [--json]
```

`--db` is required and must point at an existing cache file: a freshness check against a missing path refuses with a clear message instead of silently creating an empty sqlite file (or dying in a raw DuckDB traceback) as a side effect. `--backend` defaults to `duckdb`. `--table` restricts the report to one Odoo model; if that model isn't in the cache, the output names it, lists which models *are* cached, and prints the exact `cache_sync` command that would add it.

```
python3 -m scripts.cache_status --db ./cache.duckdb --table sale.order --json
```

detect_env — report the Claude runtime environment

One-line: detects which Claude runtime/environment the skill is running in and prints constraints and recommendations relevant to that context.

```
python3 -m scripts.detect_env [--json]
```

No required arguments; `--json` switches from human-readable text to JSON. Besides the surface, sandbox, and config-path fields, the report includes `user_profile_path` and `user_profile_exists` (whether v1.9 tailoring

context has been captured on this machine), and in headless mode its recommendations include pre-seeding the user profile with `user_profile seed --file` since the tailoring questions never run headless.

```
python3 -m scripts.detect_env --json
```

check_updates — check for a newer skill release

One-line: checks upstream for a newer Odoo Sidekick release, using a cached result unless told otherwise.

```
python3 -m scripts.check_updates [--force] [--json]
```

`--force` bypasses the cache for a fresh check. `--json` gives machine-readable output. The cached result lives at `<state_dir>/update_check.json` (following `ODOO_SIDEKICK_STATE_DIR`). Even on a cache hit, `update_available` is recomputed against the *current* local `VERSION` file rather than replayed from the cache — so after a `git pull` the tool stops announcing an update you already have, without waiting for the cache to expire or needing `--force`.

```
python3 -m scripts.check_updates
python3 -m scripts.check_updates --force --json
```

show_metrics — summarize API call metrics

One-line: summarizes Odoo API call metrics from the per-call log (counts, timing, grouping) for a session or time window.

```
python3 -m scripts.show_metrics [--since SINCE] [--last LAST]
  [--by-method] [--by-model] [--by-caller] [--json]
```

`--since` accepts a relative window (`5m` , `1h` , `2d`) or an ISO timestamp; anything else is rejected up front with a clean argparse error (exit `2`) naming both accepted forms, instead of a traceback. `--last N` limits to the last N calls. `--by-method` , `--by-model` , and `--by-caller` are **mutually exclusive** — each selects one grouped breakdown (`--by-caller` groups by the `ODOO_SIDEKICK_CALLER` identity), and passing two of them is an argument error. With no filters it defaults to the last 100 calls. The log itself is `<state_dir>/call_log.jsonl` plus rotated `call_log-*.jsonl` archives (state dir follows `ODOO_SIDEKICK_STATE_DIR`); malformed or hand-edited log lines, and lines without a parseable timestamp, are skipped rather than crashing the summary.

```
python3 -m scripts.show_metrics --since 1h --by-model
python3 -m scripts.show_metrics --last 20 --json
```

Tips and Best Practices

- **Default every profile to read-only, and only flip specific profiles to read-write when a real workflow needs it.** `mode: read-only` is already the default `scripts/profiles.py` writes for new profiles; keep it that

way unless you have a concrete write use case, and give that use case its own narrowly scoped profile rather than upgrading your everyday one.

- **Bind read-only profiles to genuinely restricted Odoo API users, not just a read-only label.** The profile's `mode` is a client-side gate, not a security boundary, since it only binds code going through this client. Run `python3 -m scripts.verify_profile <name>` after setting up any profile (and periodically thereafter) to have Odoo itself confirm what the credential can do; it exits `2` with remediation steps if a "read-only" profile is actually holding a write-capable key.
- **Keep API keys out of chat.** Load them from a `.env` file into your shell environment (`export ODOO_MAIN_API_KEY=...`) rather than pasting them into the conversation. Conversation history is long-lived across sessions, devices, and exports, even in an otherwise-ephemeral sandbox, so a key typed into chat effectively never goes away.
- **Use `--dry-run` before any consequential write.** For a `write` call it fetches the current field values and prints a before/after diff per record; for `create` it previews the (redacted) records that would be created; for `unlink` and for action/button calls it fetches and lists the target records so you can see exactly what would be affected. None of this sends anything to Odoo. Pair it with `--confirm` only once you're satisfied with the preview.
- **Check `show_metrics` after any heavy session.** `python3 -m scripts.show_metrics --since 1h` (or `--by-method`, `--by-model`, `--by-caller`, `--json`) summarizes the per-call log so you can spot retry storms, unexpectedly large read volumes, or writes you didn't expect, before they become a surprise later. On shared or multi-agent hosts, set `ODOO_SIDEKICK_CALLER` per agent so `--by-caller` breakdowns stay meaningful.
- **Treat cache staleness prompts as a real decision, not a formality.** If you're pulling from the DuckDB/SQLite cache, a stale table (especially anything older than 7 days) is flagged for a reason; refresh with `cache_sync` before trusting a report that matters.

Licensing

Odoo Sidekick is licensed under Apache 2.0 with a Commons Clause and a bespoke Competitive Use Restriction (see the `LICENSE` file for the full text). In practice: you can use it freely inside your own business, on your own Odoo, including for commercial purposes. What you can't do is resell it, wrap it into a paid hosted service, or use it to deliver Odoo/ERP consulting, implementation, or automation services to third parties if that would compete with SHIFTCollective's own services. If you're a consultancy and unsure whether your intended use crosses that line, ask before building on it.

Getting Help

Odoo Sidekick is built and maintained by **SHIFTCollective**, an AI-first consultancy. For help with the skill itself, or with your Odoo setup more broadly, reach out to:

- Email: info@shiftcollective.co
- Web: shiftcollective.co

The skill's source, and its release history, live in the upstream GitHub repository (`SHIFT-collective/Odoo-Sidekick` on `github.com`) that `scripts/check_updates.py` checks against; file issues or check released versions there.

Before reaching out, run the built-in self-check first:

```
python3 -m scripts.check_updates
```

This compares your local `VERSION` file against the upstream repository (with a 24-hour cache, so repeated calls don't hammer GitHub) and tells you whether a newer release is available, along with a link to its release notes. Pass `--force` to bypass the cache for a fresh check, or `--json` for machine-readable output. If you're debugging something odd, it's worth confirming you're on the latest version before digging further, since the bug you're hitting may already be fixed upstream.

Odoo Sidekick is built and maintained by [SHIFTcollective](#). See `LICENSE` for full licensing terms.